Appendix

Phila Haanyama (HNYPHI001) & Lulama Lingela (LNGLUL002)

Github Link: https://github.com/LlamaLuLu/EEE3096S-Practicals-Group-34.git

Code for **STM32F0:**

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2025 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include <stdlib.h>
#include <stdbool.h>

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/

/* USER CODE BEGIN PV */
//TODO: Define variables you think you might need
// - Performance timing variables (e.g execution time, throughput, pixels per second, clock cycles)
//task 1 - port from practical 1B
#define MAX_ITER 100  // or whichever value you need
#define SCALE 4096  // INITIAL FIXED-PT SCALE
#define MAX_TILE 128

uint32_t start_time = 0;
uint32_t end_time = 0;
uint32_t execution_time = 0;
uint64_t checksum = 0;
uint64_t cycles = 0;
uint32_t pixels_s = 0;
```

```
double exec_time_s = 0;
double throughput = 0;

int32_t height = 0;
int32_t width = 0;
int32_t max_iter = 0;
int32_t scale = 0; // for task 7

const int scales[] = {1000, 10000, 1000000};
const int max_iter_values[] = {100, 250, 500, 750, 1000};
const int image_dimensions[] = {128, 160, 192, 224, 256};

// for task 4
typedef struct {
    uint16_t width;
    uint16_t height;
} image_size_t;

typedef struct {
    uint16_t start_x, start_y;
    uint16_t width, height;
} tile_t;

typedef struct {
    uint16_t width, height;
    uint32_t execution_time_ms;
    uint64_t checksum;
    uint32_t throughput_pixels_per_sec;
    bool used_tiling;
    uint16_t num_tiles;
} performance_result_t;

image_size_t test_sizes[] = {
    {128, 128},    // from prac 1B
    {256, 256},    // from prac 1B
    {320, 240},    // QVGA
    {640, 480},    // VGA
    {800, 600},    // SVGA
    {1024, 768},   // XGA
    {1280, 720},   // HD
    {1920, 1080}   // Full HD
};

#define NUM_TEST_SIZES (sizeof(test_sizes)/sizeof(test_sizes[0]))

performance_result_t results[NUM_TEST_SIZES];
uint8_t result_count = 0;

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
/* USER CODE BEGIN PFP */
//TODO: Define any function prototypes you might need such as the calculate Mandelbrot function among others
uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int height, int max_iterations);
uint64_t calculate_mandelbrot_fixed_point_arithmetic_custom(int width, int height, int max_iterations, int scale);
uint64_t calculate_mandelbrot_double(int width, int height, int max_iterations);
uint64_t calculate_mandelbrot_tile_fixed_point(uint16_t tile_x, uint16_t tile_y,
                                               uint16_t tile_width, uint16_t tile_height,
                                               uint16_t total_width, uint16_t total_height,
                                               int max_iterations);

uint64_t calculate_mandelbrot_direct_fixed_point(int width, int height, int max_iterations);
bool can_allocate_image(uint16_t width, uint16_t height);
```

```
/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  /* USER CODE BEGIN 2 */

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
		// TASK 2:
//		for (int i = 0; i < 5; i++) {		// image sizes
//				width = image_dimensions[i];
//				height = image_dimensions[i];
//
//				for (int j = 0; j < 5; j++) {	// MAX_ITER values
//					max_iter = max_iter_values[j];
//
////					// --- FIXED-POINT version ---
////					start_time = HAL_GetTick();
////					checksum = calculate_mandelbrot_fixed_point_arithmetic(width, height, max_iter);
////					end_time = HAL_GetTick();
////					execution_time = end_time - start_time; // in ms
//
//					// --- DOUBLE version ---
//					start_time = HAL_GetTick();
//					checksum = calculate_mandelbrot_double(width, height, max_iter);
//					end_time = HAL_GetTick();
//					execution_time = end_time - start_time; // in ms
//
//					HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0); // blink to show progress
//					HAL_Delay(200);
```

```
//                                  }
//                          }
//
//                          while(1); // stop after one sweep


                // TASK 4: ================================ //
                for (int i = 0; i < sizeof(test_sizes)/sizeof(test_sizes[0]); i++) {
                        width  = test_sizes[i].width;
                        height = test_sizes[i].height;

                        bool use_tiling = !can_allocate_image(width, height);

                        start_time = HAL_GetTick();
                        if (!use_tiling) {
                           checksum = calculate_mandelbrot_direct_fixed_point(width, height, MAX_ITER);
                        } else {
                           checksum = 0;
                           uint16_t num_tiles_x = (width  + MAX_TILE - 1) / MAX_TILE;
                           uint16_t num_tiles_y = (height + MAX_TILE - 1) / MAX_TILE;

                           for (int ty = 0; ty < num_tiles_y; ty++) {
                              for (int tx = 0; tx < num_tiles_x; tx++) {
                                 uint16_t tile_w = ((tx+1)*MAX_TILE > width)  ? (width  - tx*MAX_TILE) : MAX_TILE;
                                 uint16_t tile_h = ((ty+1)*MAX_TILE > height) ? (height - ty*MAX_TILE) : MAX_TILE;

                                 checksum += calculate_mandelbrot_tile_fixed_point(
                                                 tx*MAX_TILE, ty*MAX_TILE,
                                                 tile_w, tile_h,
                                                 width, height, MAX_ITER);
                              }
                           }
                        }
                        end_time = HAL_GetTick();
                        execution_time = end_time - start_time; // ms
                        cycles = (uint64_t)execution_time * 48000000 / 1000; // 48 MHz clock
                        pixels_s = (uint32_t)((double)(width*height) / (execution_time / 1000.0));

                        // optional: blink LED as progress indicator
                        HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);
                        HAL_Delay(100);

                        // === RECORD RESULT ===
                        results[result_count].width  = width;
                        results[result_count].height = height;
                        results[result_count].execution_time_ms = execution_time;
                        results[result_count].checksum = checksum;
                        results[result_count].throughput_pixels_per_sec = pixels_s;
                        results[result_count].used_tiling = use_tiling;
                        results[result_count].num_tiles = use_tiling ?
                                  ((width + MAX_TILE - 1)/MAX_TILE) * ((height + MAX_TILE - 1)/MAX_TILE) : 1;

                        result_count++;
                }
                while(1); // stop after one sweep


                // TASK 5: ================================ //
//      /* USER CODE END WHILE */
//              for (int i = 0; i < 5; i++) {
//                              width = image_dimensions[i];
//                              height = image_dimensions[i];
//      /* USER CODE BEGIN 3 */
//              //TODO: Visual indicator: Turn on LED0 to signal processing start
//               HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);
//
```

```
//
//            //TODO: Benchmark and Profile Performance
//
//             // Fixed-point benchmark:
//                 //TODO: Record the start time
//                 start_time = HAL_GetTick();
//                 //TODO: Call the Mandelbrot Function and store the output in the checksum variable defined initially
//                 checksum = calculate_mandelbrot_fixed_point_arithmetic(width, height, MAX_ITER);
//                 //TODO: Record the end time
//                 end_time = HAL_GetTick();
//                 //TODO: Calculate the execution time
//                 execution_time = end_time - start_time;
//
//                 // execution time in seconds
//                 exec_time_s = execution_time / 1000;
//                 // calculate number of cycles
//                 cycles = (uint64_t)(exec_time_s*48000000);
//                 // throughput = (width*height) / exec_time_s;


//             // Double benchmark:
//                 //TODO: Record the start time
//                 start_time = HAL_GetTick();
//                 //TODO: Call the Mandelbrot Function and store the output in the checksum variable defined initially
//                 checksum = calculate_mandelbrot_double(width, height, MAX_ITER);
//                 //TODO: Record the end time
//                 end_time = HAL_GetTick();
//                 //TODO: Calculate the execution time
//                 execution_time = end_time - start_time;
//
//                 // execution time in seconds
//                 exec_time_s = execution_time / 1000.0;
//                 // calculate number of cycles
//                 cycles = (uint64_t)(exec_time_s*48000000);
//                 throughput = (width*height) / exec_time_s;

//             //TODO: Visual indicator: Turn on LED1 to signal processing start
//             HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET);
//
//
//             //TODO: Keep the LEDs ON for 2s
//     HAL_Delay(2000);
//
//             // TODO: Turn OFF LEDs
//     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0 | GPIO_PIN_1, GPIO_PIN_RESET);
//
//             }


//             // TASK 7: ============================== //
//                                 for (int i = 0; i < 5; i++) {            // image sizes
//                                         width = image_dimensions[i];
//                                         height = image_dimensions[i];
//
//                                         for (int j = 0; j < 3; j++) {
//                                                 scale = scales[j];
//
//                                                 // --- FIXED-POINT version ---
//                                                 start_time = HAL_GetTick();
//                                                 checksum = calculate_mandelbrot_fixed_point_arithmetic_custom(width,
height, MAX_ITER, scale);
//                                                 end_time = HAL_GetTick();
//                                                 execution_time = end_time - start_time; // in ms
//                                                 cycles = (uint64_t)(execution_time * 48000000 / 1000);
//                                                 pixels_s = (width * height) / (execution_time / 1000);
//
```

```
////                                            // --- DOUBLE version ---
////                                            start_time = HAL_GetTick();
////                                            checksum = calculate_mandelbrot_double(256, 256, max_iter);
////                                            end_time = HAL_GetTick();
////                                            execution_time = end_time - start_time; // in ms
//
//                                              HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0); // blink to show progress
//                                              HAL_Delay(200);
//                                        }
//                                    }
//
//                        while(1);

//          // TASK 8: ============================== //
//          HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);
//          checksum = calculate_mandelbrot_fixed_point_arithmetic(128, 128, MAX_ITER);
//          // checksum = calculate_mandelbrot_fixed_point_arithmetic(256, 256, MAX_ITER);
//          HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_1);
//          HAL_Delay(200);
//          HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_2);

  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL12;
  RCC_OscInitStruct.PLL.PREDIV = RCC_PREDIV_DIV1;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_1) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief GPIO Initialization Function
  * @param None
```

```
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOF_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
                    |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7, GPIO_PIN_RESET);

  /*Configure GPIO pins : PB0 PB1 PB2 PB3
                         PB4 PB5 PB6 PB7 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
                    |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
//TODO: Function signatures you defined previously , implement them here
        // FIXED-PT WITH DEFAULT SCALE
        uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int height, int max_iterations) {
                uint64_t mandelbrot_sum = 0;

                for(int y = 0; y < height; y++){
                        for(int x = 0; x < width; x++){
                                // convert pixel coordinates to fixed point complex plane coordinates
                                int32_t x0 = ((int64_t)x*SCALE*35/(width*10))-(SCALE*25)/10; // making sure all
numbers are integers not floats
                                int32_t y0 = ((int64_t)y * SCALE * 2/ height)-SCALE;

                                int32_t xi = 0;
                                int32_t yi = 0;
                                int i = 0;

                                while (i < max_iterations){
                                        int64_t xi_sq = ((int64_t)xi * xi) / SCALE;
                                        int64_t yi_sq = ((int64_t)yi * yi) / SCALE;

                                        if(xi_sq + yi_sq <= 4 * SCALE){
                                                int32_t temp = (int32_t)(xi_sq - yi_sq);
                                                int64_t xy = ((int64_t)xi * yi) / SCALE;

                                                yi  =(int32_t)(2*xy + y0);
                                                xi = temp + x0;

                                                i++;
                                        }

                                        else{
                                                break;
                                        }
```

```
                                        }
                                        mandelbrot_sum += i;
                                }
                        }
                        return mandelbrot_sum;
                }

                // FIXED-PT WITH CUSTOM SCALE
                uint64_t calculate_mandelbrot_fixed_point_arithmetic_custom(int width, int height, int max_iterations, int scale)
{
                        uint64_t mandelbrot_sum = 0;

                        for(int y = 0; y < height; y++){
                                for(int x = 0; x < width; x++){
                                        // convert pixel coordinates to fixed point complex plane coordinates
                                        int32_t x0 = ((int64_t)x*scale*35/(width*10))-(scale*25)/10; // making sure all
numbers are integers not floats
                                        int32_t y0 = ((int64_t)y * scale * 2/ height)-scale;

                                        int32_t xi = 0;
                                        int32_t yi = 0;
                                        int i = 0;

                                        while (i < max_iterations){
                                                int64_t xi_sq = ((int64_t)xi * xi) / scale;
                                                int64_t yi_sq = ((int64_t)yi * yi) / scale;

                                                if(xi_sq + yi_sq <= 4 * scale){
                                                        int32_t temp = (int32_t)(xi_sq - yi_sq);
                                                        int64_t xy = ((int64_t)xi * yi) / scale;

                                                        yi  =(int32_t)(2*xy + y0);
                                                        xi = temp + x0;

                                                        i++;
                                                }

                                                else{
                                                        break;
                                                }
                                        }
                                        mandelbrot_sum += i;
                                }
                        }
                        return mandelbrot_sum;
                }

                // fixed-pt Mandelbrot calculation for a single tile
                uint64_t calculate_mandelbrot_tile_fixed_point(uint16_t tile_x, uint16_t tile_y,
                                                uint16_t tile_width, uint16_t tile_height,
                                                uint16_t total_width, uint16_t total_height,
                                                int max_iterations) {
                    uint64_t mandelbrot_sum = 0;

                    for(int py = 0; py < tile_height; py++){
                        for(int px = 0; px < tile_width; px++){
                            // Calculate actual coordinates in the full image
                            int actual_x = tile_x + px;
                            int actual_y = tile_y + py;

                            // Convert pixel coordinates to fixed point complex plane coordinates
                            // Using the full image dimensions for coordinate mapping
                            int32_t x0 = ((int64_t)actual_x*SCALE*35/(total_width*10))-(SCALE*25)/10;
                            int32_t y0 = ((int64_t)actual_y * SCALE * 2/ total_height)-SCALE;

                            int32_t xi = 0;
```

```
            int32_t yi = 0;
            int i = 0;

            while (i < max_iterations){
               int64_t xi_sq = ((int64_t)xi * xi) / SCALE;
               int64_t yi_sq = ((int64_t)yi * yi) / SCALE;

               if(xi_sq + yi_sq <= 4 * SCALE){
                  int32_t temp = (int32_t)(xi_sq - yi_sq);
                  int64_t xy = ((int64_t)xi * yi) / SCALE;

                  yi = (int32_t)(2*xy + y0);
                  xi = temp + x0;

                  i++;
               } else {
                  break;
               }
            }
            mandelbrot_sum += i;
         }
      }
      return mandelbrot_sum;
   }

   uint64_t calculate_mandelbrot_direct_fixed_point(int width, int height, int max_iterations) {
      return calculate_mandelbrot_fixed_point_arithmetic(width, height, max_iterations);
   }

   // double precision implementation
   uint64_t calculate_mandelbrot_double(int width, int height, int max_iterations){
      uint64_t mandelbrot_sum = 0;

      for (int y = 0; y < height; ++y) {
                  double y0 = ((double)y / (double)height) * 2.0 - 1.0;      // map y -> [-1, +1]
                  for (int x = 0; x < width; ++x) {
                        double x0 = ((double)x / (double)width) * 3.5 - 2.5;  // map x -> [-2.5, 1.0]
                        double xi = 0.0;
                        double yi = 0.0;
                        int i = 0;

                        while (i < max_iterations && (xi*xi + yi*yi) <= 4.0) {
                              double tmp = xi*xi - yi*yi + x0;
                              yi = 2.0 * xi * yi + y0;
                              xi = tmp;
                              ++i;
                        }
                        mandelbrot_sum += (uint64_t)i;
                  }
         }
         return mandelbrot_sum;
   }

   // for task 4: test if can allocate memory for a given size
   bool can_allocate_image(uint16_t width, uint16_t height) {
         uint32_t bytes_needed = (uint32_t)width * height * sizeof(uint32_t);

         const uint32_t SRAM_LIMIT = 8 * 1024;   // 8 KB
         // const uint32_t SRAM_LIMIT = 128 * 1024; // 128 KB

         // Leave some margin for stack/heap (say 75%)
         return (bytes_needed <= (SRAM_LIMIT * 3) / 4);
   }

/* USER CODE END 4 */
```

```
/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

Code for **STM32F0**:

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2025 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include <stdlib.h>
#include <stdbool.h>

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/

/* USER CODE BEGIN PV */
//TODO: Define variables you think you might need
// - Performance timing variables (e.g execution time, throughput, pixels per second, clock cycles)

//task 1 - port from practical 1B
#define MAX_ITER 100  // INITIAL ITERATIONS VALUE
#define SCALE 4096  // INITIAL FIXED-PT SCALE
#define MAX_W 1920
#define MAX_H 1080
#define MAX_TILE 128

uint32_t start_time = 0;
uint32_t end_time = 0;
uint32_t execution_time = 0;
uint64_t checksum = 0;
uint32_t start_cycles = 0;
uint32_t elapsed_cycles = 0;
uint32_t pixels_s = 0;

int32_t height = 0;
```

```
int32_t width = 0;

int32_t max_iter = 0; // for task 2
int32_t scale = 0; // for task 7

const int scales[] = {1000, 10000, 1000000};
const int max_iter_values[] = {100, 250, 500, 750, 1000};
const int image_dimensions[] = {128, 160, 192, 224, 256};

// for task 4
typedef struct {
    uint16_t width;
    uint16_t height;
} image_size_t;

typedef struct {
    uint16_t start_x, start_y;
    uint16_t width, height;
} tile_t;

typedef struct {
    uint16_t width, height;
    uint32_t execution_time_ms;
    uint64_t checksum;
    uint32_t throughput_pixels_per_sec;
    bool used_tiling;
    uint16_t num_tiles;
} performance_result_t;

image_size_t test_sizes[] = {
    {128, 128},    // from prac 1B
    {256, 256},    // from prac 1B
    {320, 240},    // QVGA
    {640, 480},    // VGA
    {800, 600},    // SVGA
    {1024, 768},   // XGA
    {1280, 720},   // HD
    {1920, 1080}   // Full HD
};

#define NUM_TEST_SIZES (sizeof(test_sizes)/sizeof(test_sizes[0]))

performance_result_t results[NUM_TEST_SIZES];
uint8_t result_count = 0;

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
/* USER CODE BEGIN PFP */
//TODO: Define any function prototypes you might need such as the calculate Mandelbrot function among others
uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int height, int max_iterations);
uint64_t calculate_mandelbrot_fixed_point_arithmetic_custom(int width, int height, int max_iterations, int32_t scale);
uint64_t calculate_mandelbrot_double(int width, int height, int max_iterations);
uint64_t calculate_mandelbrot_float(int width, int height, int max_iterations);
uint64_t calculate_mandelbrot_tile_fixed_point(uint16_t tile_x, uint16_t tile_y,
                                               uint16_t tile_width, uint16_t tile_height,
                                               uint16_t total_width, uint16_t total_height,
                                               int max_iterations);

uint64_t calculate_mandelbrot_direct_fixed_point(int width, int height, int max_iterations);
bool can_allocate_image(uint16_t width, uint16_t height);

/* USER CODE END PFP */
```

```
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();


  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  /* USER CODE BEGIN 2 */

  // DWT cycle counter setup
  CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk; // Enable trace and debug
  DWT->CYCCNT = 0;                                // Reset counter
  DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;            // Enable cycle counter

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
//            // TASK 2: ================================= //
//            for (int i = 0; i < 5; i++) {            // image sizes
//                        width = image_dimensions[i];
//                        height = image_dimensions[i];
//
//                        for (int j = 0; j < 5; j++) {        // MAX_ITER values
//                                    max_iter = max_iter_values[j];
//
//                                    // --- FIXED-POINT version ---
//                                    start_cycles = DWT->CYCCNT;
//                                    checksum = calculate_mandelbrot_fixed_point_arithmetic(width, height, max_iter);
//                                    elapsed_cycles = DWT->CYCCNT - start_cycles;
//                                    execution_time = (double)elapsed_cycles / ((double)SystemCoreClock/1000); // in ms
//
//                                    // --- DOUBLE version ---
//                                    start_cycles = DWT->CYCCNT;
//                                    checksum = calculate_mandelbrot_double(width, height, max_iter);
//                                    elapsed_cycles = DWT->CYCCNT - start_cycles;
```

```
//                              execution_time = (double)elapsed_cycles / ((double)SystemCoreClock/1000); // in ms
//
//                              HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0); // blink to show progress
//                              HAL_Delay(200);
//                          }
//                      }
//
//                      while(1); // stop after one sweep


                        // TASK 4: ============================== //
                        for (int i = 0; i < sizeof(test_sizes)/sizeof(test_sizes[0]); i++) {
                            width  = test_sizes[i].width;
                            height = test_sizes[i].height;

                            bool use_tiling = !can_allocate_image(width, height);

                            start_cycles = DWT->CYCCNT;
                            if (!use_tiling) {
                                // direct computation if memory is OK
                                checksum = calculate_mandelbrot_direct_fixed_point(width, height, MAX_ITER);
                            } else {
                                // break into tiles
                                checksum = 0;
                                uint16_t num_tiles_x = (width  + MAX_TILE - 1) / MAX_TILE;
                                uint16_t num_tiles_y = (height + MAX_TILE - 1) / MAX_TILE;

                                for (int ty = 0; ty < num_tiles_y; ty++) {
                                    for (int tx = 0; tx < num_tiles_x; tx++) {
                                        uint16_t tile_w = ((tx+1)*MAX_TILE > width)  ? (width  - tx*MAX_TILE) : MAX_TILE;
                                        uint16_t tile_h = ((ty+1)*MAX_TILE > height) ? (height - ty*MAX_TILE) : MAX_TILE;

                                        checksum += calculate_mandelbrot_tile_fixed_point(
                                                    tx*MAX_TILE, ty*MAX_TILE,
                                                    tile_w, tile_h,
                                                    width, height, MAX_ITER);
                                    }
                                }
                            }
                            elapsed_cycles = DWT->CYCCNT - start_cycles;
                            execution_time = (uint32_t)((double)elapsed_cycles / ((double)SystemCoreClock/1000)); // ms
                            pixels_s = (uint32_t)((double)(width*height) / (execution_time / 1000.0));

                            // optional: blink LED as progress indicator
                            HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);
                            HAL_Delay(100);

                            // === RECORD RESULT ===
                            results[result_count].width  = width;
                            results[result_count].height = height;
                            results[result_count].execution_time_ms = execution_time;
                            results[result_count].checksum = checksum;
                            results[result_count].throughput_pixels_per_sec = pixels_s;
                            results[result_count].used_tiling = use_tiling;
                            results[result_count].num_tiles = use_tiling ?
                                ((width + MAX_TILE - 1)/MAX_TILE) * ((height + MAX_TILE - 1)/MAX_TILE) : 1;

                            result_count++;
                        }
                        while(1); // stop after one sweep


                        // TASK 5: ============================== //
//    /* USER CODE END WHILE */
//                      for (int i = 0; i < 5; i++) {
//                          width = image_dimensions[i];
```

```
//                height = image_dimensions[i];
//
//    /* USER CODE BEGIN 3 */
//            //TODO: Visual indicator: Turn on LED0 to signal processing start
//            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);
//
//            /* commenting out fixed-point implementation for task 5
//            //TODO: Benchmark and Profile Performance
//     // Fixed-point benchmark:
//     //TODO: Record the start number of cycles
//     start_cycles = DWT->CYCCNT;
//     //TODO: Call the Mandelbrot Function and store the output in the checksum variable defined initially
//     checksum = calculate_mandelbrot_fixed_point_arithmetic(width, height, MAX_ITER);
//     //TODO: Calculate elapsed cycles
//     elapsed_cycles = DWT->CYCCNT - start_cycles;
//     // convert cycles to seconds
//     execution_time = elapsed_cycles / SystemCoreClock; //SystemCoreClock => HAL variable that stores CPU
frequency (Hz)
//
//     */
//
//     // Double benchmark:
//     //TODO: Record the start number of cycles
//     start_cycles = DWT->CYCCNT;
//     //TODO: Call the Mandelbrot Function and store the output in the checksum variable defined initially
//     checksum = calculate_mandelbrot_double(width, height, MAX_ITER);
//     //TODO: Calculate elapsed cycles
//     elapsed_cycles = DWT->CYCCNT - start_cycles;
//     // convert cycles to seconds
//     execution_time = elapsed_cycles / SystemCoreClock;
//
//     /*
//     // floating point benchmark:
//     // record start number of cycles
//     start_cycles = DWT -> CYCCNT;
//     // call Mandelbrot Function and store the output in the checksum variable
//     checksum = calculate_mandelbrot_float(width, height, MAX_ITER);
//     // calculate elapsed cycles
//     elapsed_cycles = DWT ->CYCCNT - start_cycles;
//     //convert cycles to seconds
//     execution_time = elapsed_cycles / SystemCoreClock;
//            */
//
//            //TODO: Visual indicator: Turn on LED1 to signal processing start
//     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET);
//
//            //TODO: Keep the LEDs ON for 2s
//     HAL_Delay(2000);
//
//            //TODO: Turn OFF LEDs
//     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0 | GPIO_PIN_1, GPIO_PIN_RESET);
//            }


//            // TASK 7: ================================ //
//                    for (int i = 0; i < 5; i++) {           // image sizes
//                            width = image_dimensions[i];
//                            height = image_dimensions[i];
//
//                            for (int j = 0; j < 3; j++) {
//                                    scale = scales[j];
//
//                                    // --- FIXED-POINT version ---
//                                    start_cycles = DWT->CYCCNT;
//                                    checksum = calculate_mandelbrot_fixed_point_arithmetic_custom(width, height,
MAX_ITER, scale);
```

```
//                                      elapsed_cycles = DWT->CYCCNT - start_cycles;
//                                      execution_time = (double)elapsed_cycles / ((double)SystemCoreClock/1000); // in
ms
//                                      pixels_s = (width * height) / (execution_time / 1000);
//
////                                     // --- DOUBLE version ---
////                                     start_cycles = DWT->CYCCNT;
////                                     checksum = calculate_mandelbrot_double(width, height, MAX_ITER);
////                                     elapsed_cycles = DWT->CYCCNT - start_cycles;
////                                     execution_time = (double)elapsed_cycles / ((double)SystemCoreClock/1000); // in
ms
////                                     pixels_s = (width * height) / (execution_time * 1000);
//
//                                      HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0); // blink to show progress
//                                      HAL_Delay(200);
//                                  }
//                              }
//
//                              while(1); // stop after one sweep


//          // TASK 8: ============================== //
//          HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0);
//          checksum = calculate_mandelbrot_fixed_point_arithmetic(128, 128, MAX_ITER);
//          // checksum = calculate_mandelbrot_fixed_point_arithmetic(256, 256, MAX_ITER);
//          HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_1);
//          HAL_Delay(200);
//          HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_2);

  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage
  */
  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLM = 15;
  RCC_OscInitStruct.PLL.PLLN = 144;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
  RCC_OscInitStruct.PLL.PLLQ = 2;
  RCC_OscInitStruct.PLL.PLLR = 2;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
```

```
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_3) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
                        |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7, GPIO_PIN_RESET);

  /*Configure GPIO pins : PB0 PB1 PB2 PB3
                         PB4 PB5 PB6 PB7 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
                        |GPIO_PIN_4|GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
//TODO: Function signatures you defined previously , implement them here
            //fixed point implementation
            // MAX_ITER=4096
            uint64_t calculate_mandelbrot_fixed_point_arithmetic(int width, int height, int max_iterations) {
                        uint64_t mandelbrot_sum = 0;

                        for(int y = 0; y < height; y++){
                                    for(int x = 0; x < width; x++){
                                                // convert pixel coordinates to fixed point complex plane coordinates
                                                int32_t x0 = ((int64_t)x*SCALE*35/(width*10))-(SCALE*25)/10; // making sure all
numbers are integers not floats
                                                int32_t y0 = ((int64_t)y * SCALE * 2/ height)-SCALE;

                                                int32_t xi = 0;
                                                int32_t yi = 0;
                                                int i = 0;
```

```
                while (i < max_iterations){
                        int64_t xi_sq = ((int64_t)xi * xi) / SCALE;
                        int64_t yi_sq = ((int64_t)yi * yi) / SCALE;

                        if(xi_sq + yi_sq <= 4 * SCALE){
                                int32_t temp = (int32_t)(xi_sq - yi_sq);
                                int64_t xy = ((int64_t)xi * yi) / SCALE;

                                yi  =(int32_t)(2*xy + y0);
                                xi = temp + x0;

                                i++;
                        }

                        else{
                                break;
                        }
                }
                mandelbrot_sum += i;
            }
        }
        return mandelbrot_sum;
    }

    uint64_t calculate_mandelbrot_fixed_point_arithmetic_custom(int width, int height, int max_iterations, int32_t
scale) {
        uint64_t mandelbrot_sum = 0;

        for(int y = 0; y < height; y++){
            for(int x = 0; x < width; x++){
                // convert pixel coordinates to fixed point complex plane coordinates
                int32_t x0 = ((int64_t)x*scale*35/(width*10))-(scale*25)/10; // making sure all
numbers are integers not floats
                int32_t y0 = ((int64_t)y * scale * 2/ height)-scale;

                int32_t xi = 0;
                int32_t yi = 0;
                int i = 0;

                while (i < max_iterations){
                        int64_t xi_sq = ((int64_t)xi * xi) / scale;
                        int64_t yi_sq = ((int64_t)yi * yi) / scale;

                        if(xi_sq + yi_sq <= 4 * scale){
                                int32_t temp = (int32_t)(xi_sq - yi_sq);
                                int64_t xy = ((int64_t)xi * yi) / scale;

                                yi  =(int32_t)(2*xy + y0);
                                xi = temp + x0;

                                i++;
                        }

                        else{
                                break;
                        }
                }
                mandelbrot_sum += i;
            }
        }
        return mandelbrot_sum;
    }

    // fixed-pt Mandelbrot calculation for a single tile
    uint64_t calculate_mandelbrot_tile_fixed_point(uint16_t tile_x, uint16_t tile_y,
```

```
                                                                                uint16_t
tile_width, uint16_t tile_height,
                                                                                uint16_t
total_width, uint16_t total_height,
                                                                                int
max_iterations) {
        uint64_t mandelbrot_sum = 0;

        for(int py = 0; py < tile_height; py++){
            for(int px = 0; px < tile_width; px++){
                // Calculate actual coordinates in the full image
                int actual_x = tile_x + px;
                int actual_y = tile_y + py;

                // Convert pixel coordinates to fixed point complex plane coordinates
                // Using the full image dimensions for coordinate mapping
                int32_t x0 = ((int64_t)actual_x*SCALE*35/(total_width*10))-(SCALE*25)/10;
                int32_t y0 = ((int64_t)actual_y * SCALE * 2/ total_height)-SCALE;

                int32_t xi = 0;
                int32_t yi = 0;
                int i = 0;

                while (i < max_iterations){
                    int64_t xi_sq = ((int64_t)xi * xi) / SCALE;
                    int64_t yi_sq = ((int64_t)yi * yi) / SCALE;

                    if(xi_sq + yi_sq <= 4 * SCALE){
                        int32_t temp = (int32_t)(xi_sq - yi_sq);
                        int64_t xy = ((int64_t)xi * yi) / SCALE;

                        yi = (int32_t)(2*xy + y0);
                        xi = temp + x0;

                        i++;
                    } else {
                        break;
                    }
                }
                mandelbrot_sum += i;
            }
        }
        return mandelbrot_sum;
    }

    // Direct processing (when memory allows)
    uint64_t calculate_mandelbrot_direct_fixed_point(int width, int height, int max_iterations) {
        return calculate_mandelbrot_tile_fixed_point(0, 0, width, height, width, height, max_iterations);
    }


    //double precision implementation
    uint64_t calculate_mandelbrot_double(int width, int height, int max_iterations){
       uint64_t mandelbrot_sum = 0;
        for (int y = 0; y < height; ++y) {
            double y0 = ((double)y / (double)height) * 2.0 - 1.0;      // map y -> [-1, +1]
            for (int x = 0; x < width; ++x) {
                double x0 = ((double)x / (double)width) * 3.5 - 2.5;  // map x -> [-2.5, 1.0]
                double xi = 0.0;
                double yi = 0.0;
                int i = 0;

                while (i < max_iterations && (xi*xi + yi*yi) <= 4.0) {
                    double tmp = xi*xi - yi*yi + x0;
                    yi = 2.0 * xi * yi + y0;
                    xi = tmp;
```

```
                                        ++i;
                                }
                                mandelbrot_sum += (uint64_t)i;
                        }
                }
            return mandelbrot_sum;
        }


        //float implementation
        uint64_t calculate_mandelbrot_float(int width, int height, int max_iterations){
                uint64_t mandelbrot_sum = 0;
                for(int y = 0; y < height; y++){
                        float y0 = ((float)y / (float)height) * 2.0f - 1.0f;
                        for(int x = 0; x < width; ++x){
                                float x0 = ((float)x / (float)width) * 3.5f - 2.5f;
                                float xi = 0.0f;
                                float yi = 0.0f;
                                int i = 0;

                                while (i < max_iterations && (xi*xi* + yi*yi) <= 4.0f){
                                        float tmp = xi*xi - yi*yi + x0;
                                        yi = 2.0f * xi * yi + y0;
                                        xi = tmp;
                                        ++i;
                                }
                                mandelbrot_sum += (uint64_t)i;
                        }
                }
                return mandelbrot_sum;
        }

        // for task 4: test if can allocate memory for a given size
        bool can_allocate_image(uint16_t width, uint16_t height) {
            uint32_t bytes_needed = (uint32_t)width * height * sizeof(uint32_t);

            // const uint32_t SRAM_LIMIT = 8 * 1024;   // 8 KB
            const uint32_t SRAM_LIMIT = 128 * 1024; // 128 KB

            // Leave some margin for stack/heap (say 75%)
            return (bytes_needed <= (SRAM_LIMIT * 3) / 4);
        }

/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
```

```
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```